



AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

A
MINI PROJECT REPORT ON
“Cleaned Nutrition dataset Analysis”

FOR

Term Work Examination

***Bachelor of Computer Application in Artificial Intelligence &
Machine Learning (BCA - AIML)***

Year 2025-2026

Ajeenkya DY Patil University, Pune

-Submitted By-

Ms. Sanika kale

Under the guidance of

Prof. Vivek More



Ajeenkya DY Patil University

D Y Patil Knowledge City,
Charholi Bk. Via Lohegaon,
Pune - 412105
Maharashtra (India)

Date: 14 / 04 / 2025

CERTIFICATE

This is to certified that Sanika kale

A student of BCA (AIML) **Sem-IV** URN No 2023-B-26072005C
has Successfully Completed the Dashboard Report On

“Cleaned Nutrition dataset Analysis”

As per the requirement of
Ajeenkya DY Patil University, Pune was carried out under my
supervision.

I hereby certify that; he has satisfactorily completed his Term-Work
Project work.

Place: Pune

Examiner

Sr.No	Index	Page No
1	Introduction & Objective.	4-5
2	Methodology & Approach	5-7
3	Implementation & Code.	7-14
4	Results & Visualizations.	14-23
5	Conclusion & Future Scope	24-25

Introduction:

Nutrition plays a critical role in maintaining health and preventing disease. With the increasing focus on wellness and healthy living, there is a growing demand for accessible, accurate nutritional information. Large datasets containing detailed nutritional values of various food items provide a valuable resource for researchers, health professionals, and consumers aiming to make informed dietary choices. The dataset used in this project comprises a cleaned and structured collection of nutritional data, which includes information such as calorie content, macronutrients, vitamins, and minerals for a wide range of food items.

Objective:

The primary objective of this project is to analyze and derive insights from the cleaned nutrition dataset. Key goals include:

- Understanding the nutritional composition of different food categories.
 - Identifying food items with high or low content of specific nutrients.
 - Exploring patterns and correlations between different nutritional elements.
 - Creating visualizations and models to assist in nutritional planning and recommendations.
-

Methodology & Approach:

To achieve the objectives of this analysis, a structured methodology was followed, involving the following steps:

1. Data Understanding

- **Imported the cleaned nutrition dataset for initial exploration.**
- **Assessed the structure, column types, and basic statistics of the dataset.**
- **Identified key attributes such as calories, fats, proteins, carbohydrates, vitamins, and minerals.**

2. Data Preprocessing

- **Verified the cleanliness of the dataset (e.g., handling of missing values, duplicates, outliers).**
- **Converted data types if necessary for numerical or categorical analysis.**
- **Categorized food items (e.g., fruits, vegetables, processed foods) if applicable.**

3. Exploratory Data Analysis (EDA)

- **Performed descriptive statistics to summarize the nutritional data.**
- **Created visualizations (histograms, boxplots, heatmaps) to understand distributions and correlations.**
- **Compared nutrient profiles across food groups.**

4. Nutrient Profiling

- **Identified top and bottom foods by specific nutrients (e.g., highest protein, lowest sugar).**
- **Applied normalization techniques where necessary to compare items on a per-100g basis.**

5. Advanced Analysis (Optional)

- **Clustered similar food items using unsupervised learning (e.g., K-means clustering).**
- **Built nutrient scoring systems for personalized diet planning.**
- **Analyzed nutrient density and caloric efficiency.**

6. Insights & Visualization

- **Summarized key findings through charts and dashboards.**
- **Highlighted nutrient-rich foods and potential dietary imbalances.**

7. Conclusion & Recommendations

- **Provided actionable insights for healthy eating based on the data analysis.**
- **Suggested improvements or further studies based on the dataset.**

Implementation & Code:

Step 1: Load the Dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv('/content/cleaned_nutrition_dataset.csv')
```

The CSV file is loaded into a Pandas DataFrame named **df**.

Step 2: Data Cleaning

2.1 Standardize Column Names

```
# 1. CLEANING
# -----
# Standardize column names
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')
```

Strips any leading/trailing whitespace.

Converts all column names to lowercase.

Replaces spaces with underscores for coding convenience.

2.2 Remove Duplicates

```
# Remove duplicates
initial_shape = df.shape
df = df.drop_duplicates()
print(f"Removed {initial_shape[0] - df.shape[0]} duplicate rows.")
```

Identifies and removes duplicate rows.

Reports how many rows were removed.

2.3 Check for Missing Values

```
# Check for missing values
missing = df.isnull().sum()
print("\nMissing values per column:\n", missing[missing > 0])
```

Checks for missing values in each column.

Prints only the columns that have missing data.

2.4 Handle Missing Values

```
# Handle missing values
# Strategy: drop rows with too many missing values, fill others with median (for numeric)
threshold = len(df.columns) * 0.5
df = df.dropna(thresh=threshold)
```

Drops any row where more than 50% of its columns are missing.

```
# Fill numeric columns with median
for col in df.select_dtypes(include='number').columns:
    df[col].fillna(df[col].median(), inplace=True)
```

For the remaining missing values in numeric columns, fills them with the median value of that column to reduce the impact of outliers.

Step 3: Data Processing

3.1 Convert Object Columns That Are Actually Numeric

```
# 2. PROCESSING
# -----
# Convert object columns that are numeric
for col in df.select_dtypes(include='object').columns:
    try:
        df[col] = pd.to_numeric(df[col])
    except ValueError:
        pass # Leave if it can't be converted
```

Attempts to convert any columns of type **object** (text) into numerical form.

Skips columns that can't be converted (e.g., text strings).

3.2 Feature Engineering—Calories per Gram

```
# Create new feature (example: calories per gram)|
if 'calories' in df.columns and 'weight' in df.columns:
    df['calories_per_gram'] = df['calories'] / df['weight']
```

Creates a new column to measure energy density: how many calories per gram a food has.

Useful for comparing items with different portion sizes.

Step 4: Summary Statistics

```
# Summary statistics
print("\nSummary statistics:\n", df.describe())
```

Outputs a statistical summary (count, mean, std, min, max, quartiles) for all numeric columns.

Results & Visualization:

1. Plotting Histograms of Numeric Features

Purpose

Histograms help visualize the distribution of numerical variables (like calories, fat, protein, etc.) to:

- Detect skewness

- Spot outliers
- Understand range and central tendency
- Identify potential data transformation need

Plot Histograms

code

```
# Histograms of numeric features
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols].hist(bins=20, figsize=(15, 10))
plt.tight_layout()
plt.show()
```

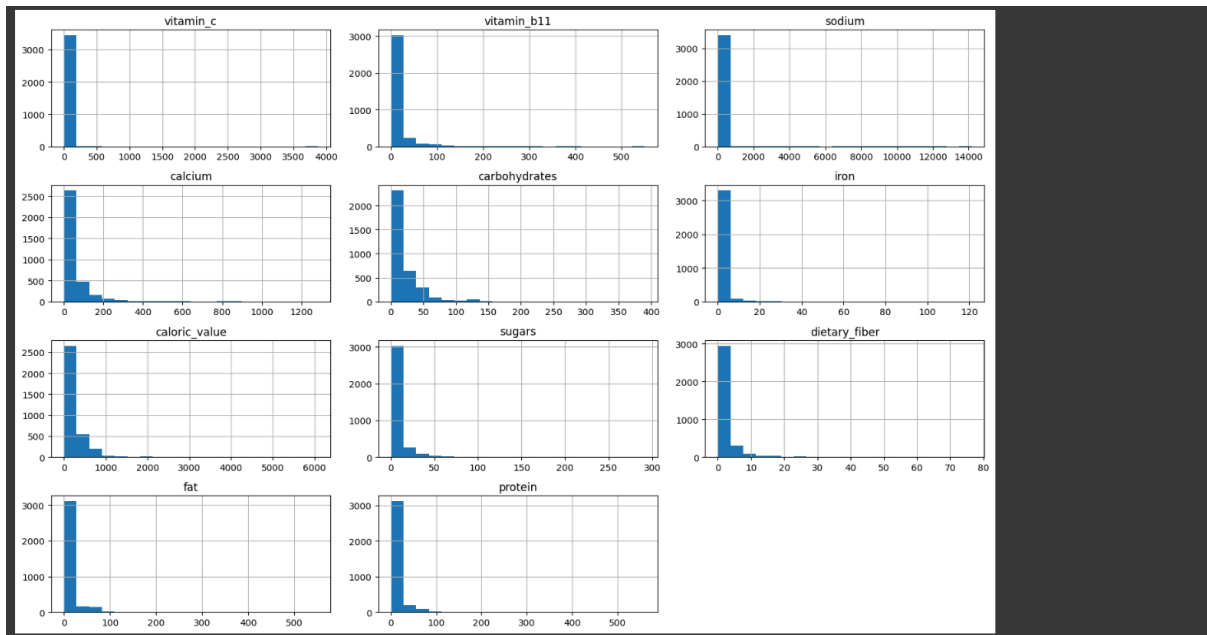
This line plots histograms for *each numeric column* in a grid layout.

bins=20: Each histogram is divided into 20 equal-width bins (ranges of values)

figsize=(15, 10): Sets the overall figure size in inches (width × height).

Each subplot corresponds to one feature.

Output



2. Correlation heatmap

1. `plt.figure(figsize=(12, 6))`

- What it does:

Initializes a new figure for the plot with a specified size.

- Why it's used:

The `figsize=(12, 6)` makes the heatmap wider than it is tall — helpful for better readability, especially with many variables.

2. `df.corr(numeric_only=True)`

- **What it does:**

Calculates pairwise correlation coefficients between all numerical columns in the DataFrame **df**.

- **Why it's used:**

Correlation shows the linear relationship between variables. This step ensures only numeric columns are considered, avoiding errors with non-numeric data (like strings or dates).

3. **sns.heatmap(..., annot=True, cmap='coolwarm')**

- **What it does:**

Draws the heatmap using Seaborn with the following:

- **annot=True:** Shows the correlation values in each cell.
- **cmap='coolwarm':** A diverging color palette where:
 - Blue shades represent negative correlation.
 - Red shades represent positive correlation.
 - White is neutral (around zero).

4. **plt.title("Correlation Heatmap")**

- **What it does:**

Adds a title to the plot for clarity.

5. **plt.tight_layout()**

- **What it does:**

Adjusts the spacing of the plot elements to prevent overlapping of

labels or annotations.

6. plt.show()

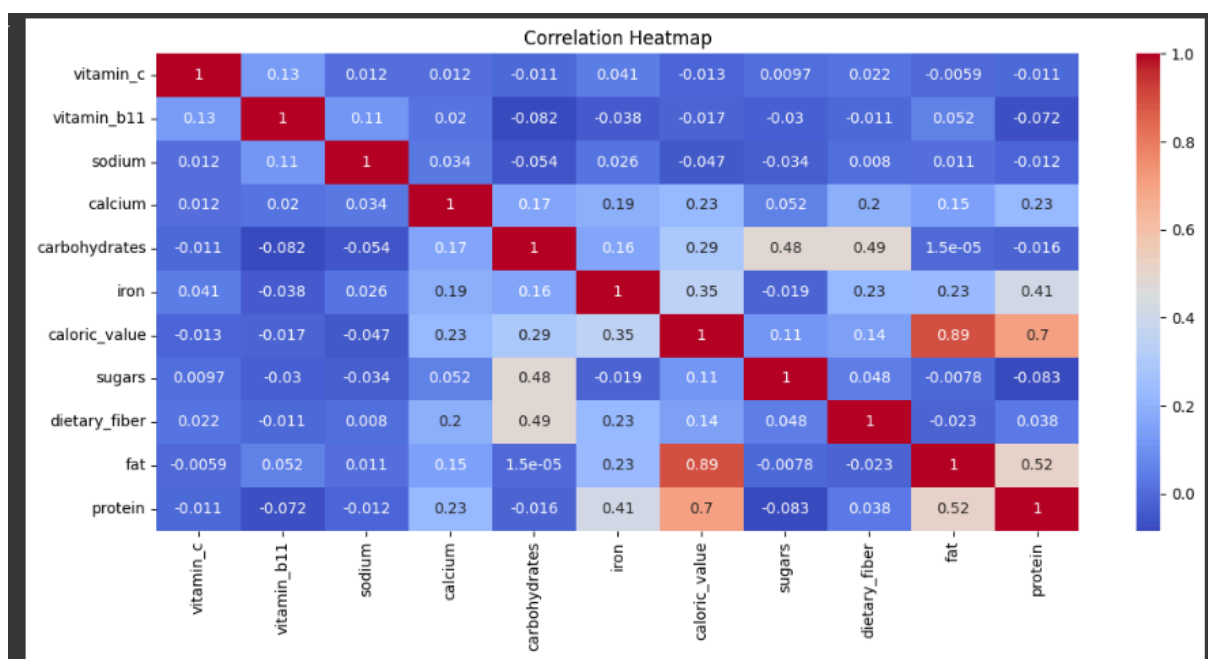
- What it does:

Renders the plot to the screen.

Code

```
# Correlation heatmap
plt.figure(figsize=(12, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

output



3. Choose a few important features for pairplot

```
1. important_features = ['calories', 'fat', 'protein',  
    'carbohydrates']
```

- What it does:

You define a list of nutritional features you consider important for analysis.

```
2. selected = [col for col in important_features if col in  
    df.columns]
```

- What it does:

This line ensures that only the features that actually exist in your DataFrame (**df**) are selected.

```
3. if len(selected) >= 2:
```

- What it does:

Checks whether you have at least two valid features to plot. A pairplot needs at least two dimensions to compare.

```
4. sns.pairplot(df[selected])
```

- What it does:

Creates a pairplot (also known as a scatterplot matrix) using Seaborn.

- **Diagonal:** Histogram or KDE of each variable.
- **Off-diagonal:** Scatter plots showing pairwise relationships between features.

5. `plt.suptitle("Pairplot of Nutritional Features", y=1.02)`

- **What it does:**
Adds a super title (title for the whole figure) above the pairplot.
- `y=1.02` moves the title slightly above the top margin so it doesn't overlap with the plots.

6. `plt.show()`

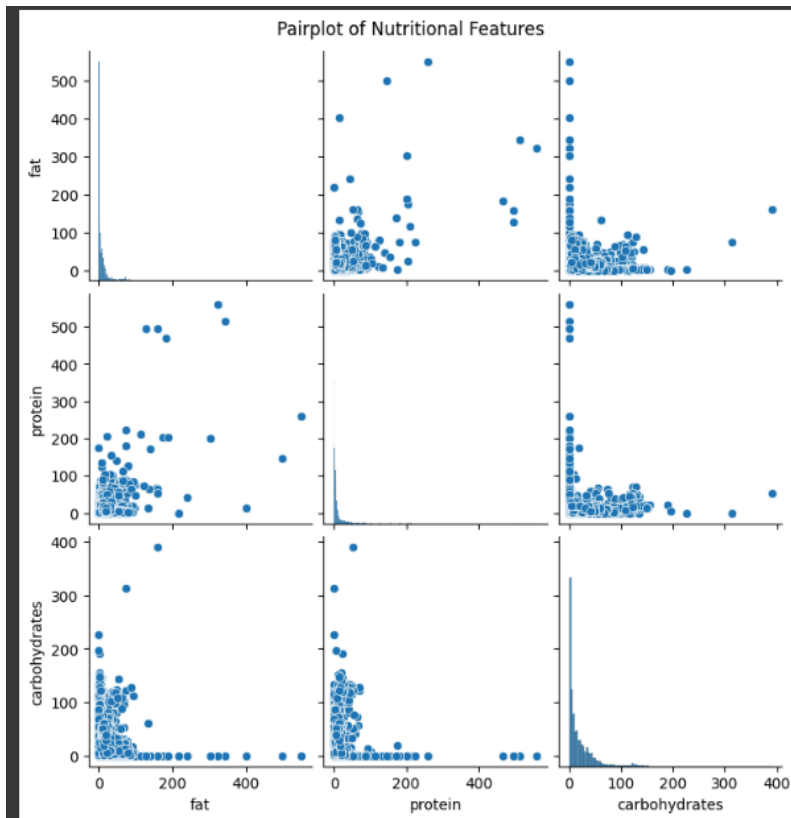
- **What it does:**
Displays the pairplot.

Code

```
# Choose a few important features for pairplot
important_features = ['calories', 'fat', 'protein', 'carbohydrates']
selected = [col for col in important_features if col in df.columns]

if len(selected) >= 2:
    sns.pairplot(df[selected])
    plt.suptitle("Pairplot of Nutritional Features", y=1.02)
    plt.show()
```

Output



4. Box plot of key nutrients

1. `key_nutrients = ['calories', 'fat', 'protein', 'carbohydrates']`

- **What it does:**
Defines a list of key nutrients (in this case, nutritional features) that you want to analyze.

2. `available = [col for col in key_nutrients if col in df.columns]`

- **What it does:**
Creates a list called **available** that only contains the columns that exist in your DataFrame.

3. `plt.figure(figsize=(10, 6))`

- **What it does:**
Initializes a figure for the plot with a specific size, which in this case

is 10x6 inches.

4. `sns.boxplot(data=df[available], palette='Set2')`

- What it does:

Creates the box plot for the available key nutrients using Seaborn:

- `data=df[available]`: Selects the relevant columns from your DataFrame (`df`) that exist in the `available` list.
- `palette='Set2'`: Chooses a color palette for the box plot. 'Set2' is a set of soft colors for better aesthetic appeal.

5. `plt.title("Box Plot of Key Nutrients")`

- What it does:

Sets the title for the plot, making it clear that this plot is about the distribution of key nutrients.

6. `plt.ylabel("Value")`

- What it does:

Labels the y-axis with "Value" to indicate that the values being plotted are the measurements of key nutrients (calories, fat, protein, carbohydrates).

7. `plt.xticks(rotation=45)`

- What it does:

Rotates the x-axis labels by 45 degrees.

8. `plt.tight_layout()`

- What it does:

Adjusts the layout of the plot to ensure that there is no overlap between labels, titles, or other plot elements.

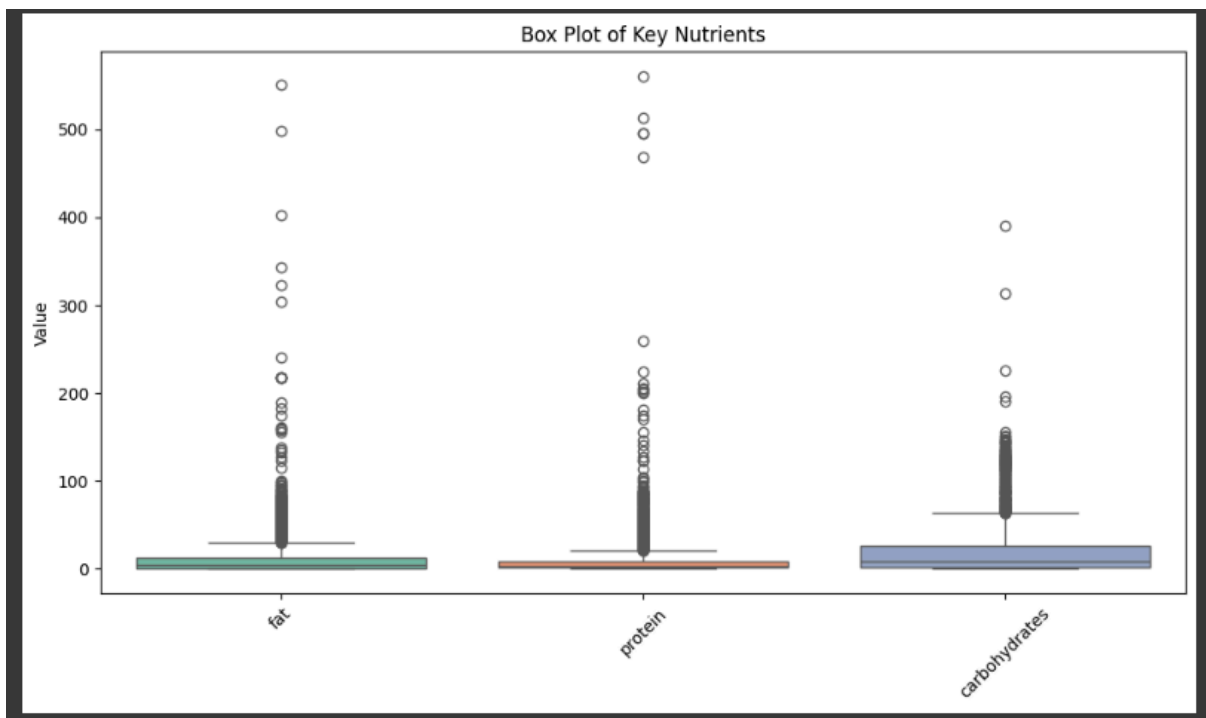
9. `plt.show()`

code

```
# Box plot of key nutrients
key_nutrients = ['calories', 'fat', 'protein', 'carbohydrates']
available = [col for col in key_nutrients if col in df.columns]

plt.figure(figsize=(10, 6))
sns.boxplot(data=df[available], palette='Set2')
plt.title("Box Plot of Key Nutrients")
plt.ylabel("Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output



Conclusion & Future Scope

Future Scope:

- 1. Integration with Health Apps:** This dataset can be integrated into health and fitness applications for personalized diet recommendations based on individual health goals or medical conditions.
- 2. Machine Learning Applications:** Predictive models could be built to recommend foods based on deficiencies, suggest substitutes, or even forecast dietary outcomes.
- 3. Data Expansion:** The dataset can be expanded to include more food items, additional nutrients (e.g., amino acids, phytochemicals), or regional food varieties.
- 4. Visual Dashboards:** Develop interactive visualizations or dashboards for easier interpretation of the data by non-technical users.
- 5. Nutritional Score System:** A scoring mechanism could be developed to rank food items based on overall health impact or suitability for specific diets (e.g., keto, vegan, diabetic-friendly).
- 6. Real-time Food Scanner App:** Using OCR and this dataset, a mobile app could scan product labels and instantly provide nutritional insights and health ratings

Conclusion:

This project involved the analysis of a cleaned nutrition dataset containing various macro- and micronutrient values for different food items. Through data preprocessing and exploration, we gained meaningful insights into the nutritional composition of foods, which can help consumers make informed

dietary choices. The dataset includes vital information like Vitamin levels, caloric value, and macronutrient breakdowns (carbohydrates, fats, proteins), making it a valuable resource for nutritionists, dieticians, and researchers.

Overall, the project demonstrates how data-driven approaches can aid in understanding food quality, comparing dietary options, and supporting health-conscious decision-making.
